Universidad Autónoma del Estado de Hidalgo

REPORTE DE PRÁCTICA NO. 6

ANÁLISIS SINTÁCTICO DE EXPRESIONES ARITMÉTICAS

ALUMNO: Samuel Ortiz Salinas
Docente: Dr. Eduardo Cornejo-Velázquez

1. Introducción

Después del análisis léxico, que agrupa los caracteres en tokens, el compilador necesita comprobar que esos tokens estén acomodados según las reglas del lenguaje. De eso se encarga el análisis sintáctico: toma la secuencia de tokens, la compara con una gramática y, si es válida, construye el árbol sintáctico que la reconoce; si no, reporta el error [1]. En esta práctica se construyeron un analizador léxico con Lex (Flex) y un analizador sintáctico con Yacc (Bison) que, juntos, validan expresiones aritméticas (con números enteros y decimales, identificadores, paréntesis y las operaciones de suma, resta, multiplicación y división) y expresiones condicionales con comparaciones.
Para probarlos se escribieron dos archivos de texto: uno con expresiones correctas y otro con expresiones que incurren en errores sintácticos y léxicos, para comprobar que el analizador acepta unas y rechaza las otras sin detenerse.

2. Marco teórico

Gramáticas libres de contexto

Una gramática formal es una tupla $G = (\Sigma_T, \Sigma_N, S, P)$ donde Σ_T es el alfabeto de símbolos terminales, Σ_N el de no terminales, $S \in \Sigma_N$ el símbolo inicial y P el conjunto de producciones; el lenguaje que genera son las cadenas de terminales que se pueden derivar desde S [3]. Las expresiones regulares solo alcanzan para los lenguajes regulares (tipo 3), y por eso no pueden verificar cosas como que los paréntesis estén balanceados; para eso se necesitan las gramáticas libres de contexto (tipo 2) [2]. La notación BNF es la forma estándar de escribir sus producciones.

Análisis sintáctico y árbol sintáctico

El analizador sintáctico recibe los tokens del analizador léxico y determina su estructura; en teoría su salida es el árbol sintáctico, pero en la práctica también detecta errores y se comunica con la tabla de símbolos [1]. Una derivación por la izquierda expande siempre el no terminal más a la izquierda, y una por la derecha el más a la derecha; el análisis ascendente (*bottom-up*), que es el que usa Yacc, reconstruye en reversa una derivación por la derecha [1].

Precedencia y asociatividad

Una gramática ambigua es aquella en la que una misma cadena tiene más de un árbol. Para que `2 * 3 + 4` se agrupe como `(2 * 3) + 4` hay que fijar la prioridad de los operadores (el $*$ y el $/$ pesan más que el $+$ y el $-$) y su asociatividad, ya sea escribiendo la gramática por niveles o declarando la precedencia en Yacc [4].

Lex y Yacc

Lex (Flex) genera el analizador léxico a partir de expresiones regulares, y Yacc (Bison) genera un analizador sintáctico LALR(1) a partir de las producciones de la gramática, con una acción en C por cada regla. Se comunican mediante la función `yylex()`, que devuelve el siguiente token, y la variable `yylval`, que lleva su valor. Para los errores, Yacc llama a `yyerror()` y ofrece el token especial `error` para recuperarse y seguir analizando [4].

3. Objetivos

Objetivo general

Desarrollar un analizador léxico y un analizador sintáctico utilizando Lex y Yacc para reconocer y validar operaciones aritméticas, asegurando la detección de errores y la correcta validación de las gramáticas definidas.

Objetivos específicos

- Emplear autómatas y gramáticas libres de contexto para definir cómo lograr el objetivo de la práctica.
- Diseñar un programa en Lex que identifique números enteros y decimales, y los símbolos de suma, resta, multiplicación y división como en Python.
- Implementar un programa en Yacc que verifique la correcta construcción de las operaciones aritméticas mediante una gramática formal.

4. Herramientas empleadas

1. **Flex:** generador de analizadores léxicos (versión libre de Lex).
2. **Bison (modo Yacc, `bison -y`):** generador de analizadores sintácticos LALR(1) a partir de una gramática.
3. **GCC:** compilador de C para unir `lex.yy.c` y `y.tab.c`.
4. **Terminal en Linux (CachyOS):** para ejecutar el analizador sobre los archivos de prueba.
5. **LaTeX (pdflatex):** para redactar este reporte con la plantilla de la materia.

5. Desarrollo

Autómatas de los tokens

Los tokens que reconoce el analizador léxico se describieron primero con autómatas finitos:

- **Entero:** estado inicial → dígito → estado final con un ciclo sobre dígitos.
- **Decimal:** entero → punto → dígito → estado final con un ciclo sobre dígitos (es obligatorio al menos un dígito después del punto).
- **Identificador:** (letra o _) → estado final con un ciclo sobre (letra, dígito o _).
- **Operadores:** un estado final por símbolo (`+ - * / ( )`); en los de dos caracteres (`**`, `==`, `!=`, `<=`, `>=`) el primer carácter lleva a un estado intermedio que, si recibe el segundo, pasa al final. Los de comparación `<` y `>` también son finales por sí solos.

En el Cuadro 1 se resumen los tokens, su expresión regular y el nombre con que se envían al analizador sintáctico.

Table 1: Tokens reconocidos por el analizador léxico.

Token	Expresión regular	Ejemplos
`NUM` (entero)	`[0-9]+`	5, 120
`NUM` (decimal)	`[0-9]+\.[0-9]+`	3.14, 0.5
`ID`	`[a-zA-Z_][a-zA-Z0-9_]*`	x, total_1
Aritméticos	`+ - * / **`	`+`, `**`
Comparación	`== != < > <= >=`	`>=`
Paréntesis	`( )`	

Gramática en notación BNF

La gramática se escribió para que la multiplicación y la división tengan más prioridad que la suma y la resta, y las comparaciones la menor de todas. La potencia (`**`, como en Python) es la de mayor prioridad y asocia por la derecha:

Listing 1: Gramática en BNF.

```
<linea>       ::= <expresion> "\n"
<expresion>   ::= <expresion> <comp> <aritmetica> | <aritmetica>
<comp>        ::= "==" | "!=" | "<" | ">" | "<=" | ">="
<aritmetica>  ::= <aritmetica> "+" <termino>
                | <aritmetica> "-" <termino>
                | <termino>
<termino>     ::= <termino> "*" <factor>
                | <termino> "/" <factor>
                | <factor>
<factor>      ::= <base> "**" <factor> | <base>
<base>        ::= NUM | ID | "-" <base> | "(" <expresion> ")"
```

Como es más cómodo y corto, en el archivo de Yacc se escribió la versión ambigua (una sola regla `expresion` con todos los operadores) y la ambigüedad se resolvió con las declaraciones `%left`, `%right` y `%prec`, de menor a mayor prioridad: comparaciones, `+ -`, `* /`, menos unario y `**`. Con eso Yacc genera el mismo análisis que la gramática por niveles de arriba. Una diferencia con lo que pedía el manual es que ahí se indica que `+` y `*` asocian por la derecha; aquí asocian por la izquierda, como en Python. En la suma y la multiplicación el resultado es el mismo, y en la resta y la división asociar por la derecha daría resultados incorrectos (`8 - 3 - 2` debe dar 3, no 7).

Árbol sintáctico

La Figura 1 muestra el árbol que se obtiene para `2 * 3 + 4`: el producto queda más abajo que la suma, por lo que se calcula primero.

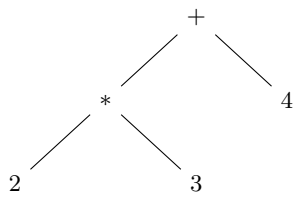


Figure 1: Árbol sintáctico de `2 * 3 + 4`.

Analizador léxico (Lex)

En el Listado 2 se presenta el programa en Lex. Convierte cada lexema en un token y entrega el valor numérico en `yylval`. Ignora espacios y comentarios que empiezan con `#` (como en Python), cuenta las líneas, y reporta como error léxico un identificador que empieza con dígito (`2valor`) o cualquier carácter desconocido (`$`, `@`).

Listing 2: Analizador léxico (analizador.l).

```
%{
#include <stdio.h>
#include <stdlib.h>
#define YYSTYPE double
#include "y.tab.h"

int linea = 1;
int tok_linea = 1;   /* linea del ultimo token leido */

int linea_mostrada = 0;
extern char *lineas[];
extern int n_lineas;

/* Antes de cada token: guarda la linea y, si es la primera vez que se ve
   la linea, imprime su texto para identificar la salida. */
#define YY_USER_ACTION \
    tok_linea = linea; \
    if (yytext[0] != '\n' && linea_mostrada != linea) { \
        linea_mostrada = linea; \
        if (linea <= n_lineas) printf("[%d] %s\n", linea, lineas[linea-1]); \
    }
%}

%option noyywrap

DIGITO  [0-9]
LETRA   [a-zA-Z_]
ID      {LETRA}({LETRA}|{DIGITO})*

%%

{DIGITO}+\.{DIGITO}+    { yylval = atof(yytext); return NUM; }
{DIGITO}+               { yylval = atof(yytext); return NUM; }
{DIGITO}+{ID}           { printf("  [Error lexico] linea %d: identificador mal formado '%s'\
    n", linea, yytext); return ERROR_LEX; }
{ID}                    { return ID; }

"**"                    { return POT; }
"=="                    { return IGUAL; }
"!="                    { return DIST; }
"<="                    { return MENIGUAL; }
```

```
">="                    { return MAYIGUAL; }
"<"                     { return '<'; }
">"                     { return '>'; }
"+"                     { return '+'; }
"-"                     { return '-'; }
"*"                     { return '*'; }
"/"                     { return '/'; }
"("                     { return '('; }
")"                     { return ')'; }

\n                      { linea++; return '\n'; }
[ \t\r]+                ;
"#".*                   ;
.                       { printf("  [Error lexico] linea %d: caracter no reconocido '%s'\n",
     linea, yytext); return ERROR_LEX; }

%%
```

Analizador sintáctico (Yacc)

En el Listado 3 se presenta el programa en Yacc. Cada línea del archivo se toma como una sentencia. Si es una expresión válida, la acción imprime `VALIDA` y su resultado (si tiene identificadores no se evalúa porque no tienen valor, y si es una comparación se muestra `True` o `False`). Si la línea tiene un error se usa la regla `error '\n'`: Yacc descarta los tokens hasta el siguiente salto de línea, se imprime `INVALIDA` y el análisis sigue con la línea siguiente, para detectar todos los errores en una sola ejecución.

Listing 3: Analizador sintáctico (analizador.y).

```
%{
#include <stdio.h>
#include <math.h>
#include <string.h>
#include <stdlib.h>

#define YYSTYPE double

extern int linea, tok_linea;
extern FILE *yyin;
int yylex(void);
void yyerror(const char *s);

int n_validas = 0, n_errores = 0;
int es_condicion = 0;   /* 1 si la linea tiene operador de comparacion */
int hay_id = 0;
char *lineas[1000];
int n_lineas = 0;         /* 1 si la linea usa identificadores (no se evalua) */
%}

%token NUM ID ERROR_LEX
%token IGUAL DIST MENIGUAL MAYIGUAL POT

%left IGUAL DIST '<' '>' MENIGUAL MAYIGUAL
%left '+' '-'
%left '*' '/'
%right UMINUS
%right POT

%%

entrada:
      /* vacio */
    | entrada linea
    ;

linea:
      '\n'
```

```
    | expresion '\n'
        {
          n_validas++;
          if (hay_id)       printf("  -> VALIDA (usa identificadores, no se evalua)\n");
          else if (es_condicion) printf("  -> VALIDA (condicion = %s)\n", $1 ? "True" : "
              False");
          else              printf("  -> VALIDA (resultado = %g)\n", $1);
          es_condicion = 0; hay_id = 0;
        }
    | error '\n'
        {
          n_errores++;
          printf("  -> INVALIDA (error sintactico en la linea %d)\n", tok_linea);
          es_condicion = 0; hay_id = 0;
          yyerrok;
        }
    ;

expresion:
      NUM                       { $$ = $1; }
    | ID                        { hay_id = 1; $$ = 0; }
    | expresion '+' expresion   { $$ = $1 + $3; }
    | expresion '-' expresion   { $$ = $1 - $3; }
    | expresion '*' expresion   { $$ = $1 * $3; }
    | expresion '/' expresion   {
                                  if ($3 == 0 && !hay_id) { printf("  [Aviso] division entre
                                      cero\n"); $$ = 0; }
                                  else $$ = ($3 == 0) ? 0 : $1 / $3;
                                }
    | expresion POT expresion   { $$ = pow($1, $3); }
    | '-' expresion %prec UMINUS { $$ = -$2; }
    | '(' expresion ')'         { $$ = $2; }
    | expresion IGUAL expresion    { es_condicion = 1; $$ = ($1 == $3); }
    | expresion DIST expresion     { es_condicion = 1; $$ = ($1 != $3); }
    | expresion '<' expresion      { es_condicion = 1; $$ = ($1 <  $3); }
    | expresion '>' expresion      { es_condicion = 1; $$ = ($1 >  $3); }
    | expresion MENIGUAL expresion { es_condicion = 1; $$ = ($1 <= $3); }
    | expresion MAYIGUAL expresion { es_condicion = 1; $$ = ($1 >= $3); }
    ;

%%

void yyerror(const char *s) {
    printf("  [Error] linea %d: %s\n", tok_linea, s);
}

int main(int argc, char **argv) {
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) { perror(argv[1]); return 1; }
        /* se guardan las lineas para mostrarlas junto con su resultado */
        char buf[512];
        while (fgets(buf, sizeof buf, yyin) && n_lineas < 1000) {
            buf[strcspn(buf, "\n")] = 0;
            lineas[n_lineas++] = strdup(buf);
        }
        rewind(yyin);
    }
    yyparse();
    printf("\n--- Resumen ---\n");
    printf("Expresiones validas  : %d\n", n_validas);
    printf("Expresiones invalidas: %d\n", n_errores);
    return 0;
}
```

Para generar y ejecutar el programa:

```
bison -y -d analizador.y
```

```
flex -o lex.yy.c analizador.l
gcc lex.yy.c y.tab.c -o analizador -lm
./analizador correctas.txt
./analizador incorrectas.txt
```

6. Resultados

Archivo con expresiones correctas

El archivo `correctas.txt` tiene 13 expresiones: operaciones con enteros y decimales, paréntesis, menos unario, potencia, identificadores y comparaciones.

Listing 4: Archivo correctas.txt.

```
2 + 3 * 4
(2 + 3) * 4
10 / 4
3.14 * 2 * 2
-5 + 8
2 ** 3 ** 2
(1.5 + 2.5) * (3 - 1)
x + y * 2
(a + b) * (c - d) / 2
10 >= 5 + 2
3 * 4 == 12
(x + 1) < (y * 2)
8 / (4 - 2) != 4
```

En la Figura 2 se muestra la salida. Las 13 se reconocieron como válidas. Se puede ver que la prioridad de los operadores funciona: `2 + 3 * 4` da 14 y `(2 + 3) * 4` da 20; que `**` asocia por la derecha (`2 ** 3 ** 2` da 512, es decir 2^9, y no 64); y que las comparaciones se evalúan después de la aritmética (`10 >= 5 + 2` es `True`).

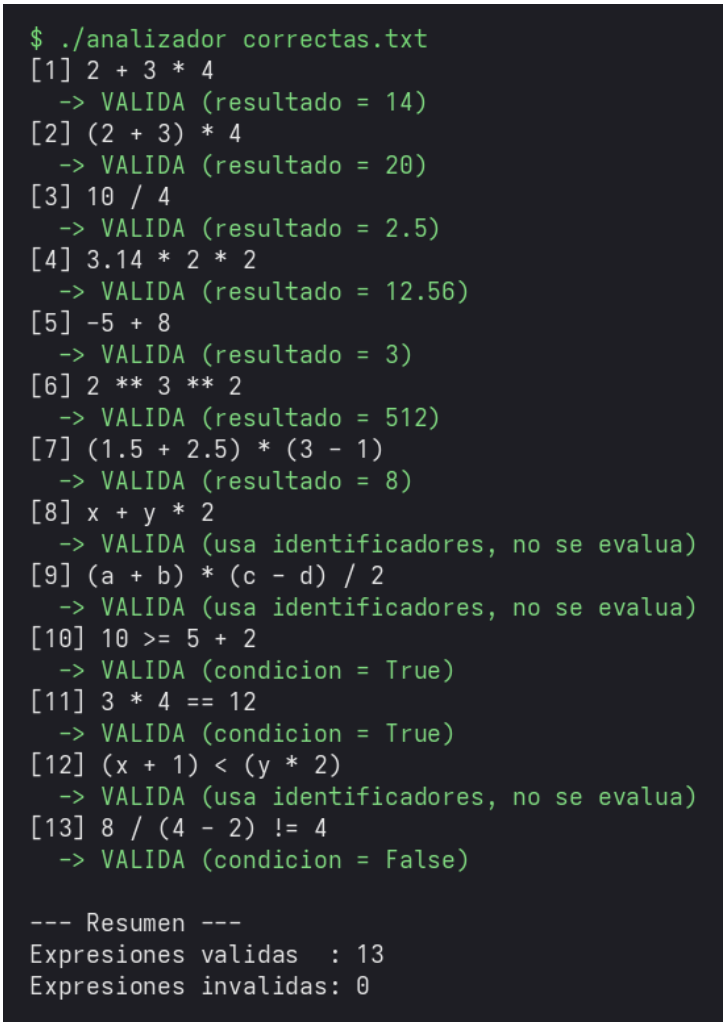


Figure 2: Salida del analizador sobre correctas.txt.

Archivo con expresiones incorrectas

El archivo `incorrectas.txt` tiene 10 expresiones con errores a propósito:

Listing 5: Archivo incorrectas.txt.

```
2 + * 3
(2 + 3 * 4
2 + 3) * 4
5 5
* 7 + 1
3 +
4 $ 2
2valor + 1
10 / ()
x + (y * ) - 2
```

En la Figura 3 se muestra la salida: las 10 se marcaron como inválidas y el analizador no se detuvo en ninguna. En el Cuadro 2 se explica la causa de cada una.

```
$ ./analizador incorrectas.txt
[1] 2 + * 3
  [Error] linea 1: syntax error
  -> INVALIDA (error sintactico en la linea 1)
[2] (2 + 3 * 4
  [Error] linea 2: syntax error
  -> INVALIDA (error sintactico en la linea 2)
[3] 2 + 3) * 4
  [Error] linea 3: syntax error
  -> INVALIDA (error sintactico en la linea 3)
[4] 5 5
  [Error] linea 4: syntax error
  -> INVALIDA (error sintactico en la linea 4)
[5] * 7 + 1
  [Error] linea 5: syntax error
  -> INVALIDA (error sintactico en la linea 5)
[6] 3 +
  [Error] linea 6: syntax error
  -> INVALIDA (error sintactico en la linea 6)
[7] 4 $ 2
  [Error lexico] linea 7: caracter no reconocido '$'
  [Error] linea 7: syntax error
  -> INVALIDA (error sintactico en la linea 7)
[8] 2valor + 1
  [Error lexico] linea 8: identificador mal formado '2valor'
  [Error] linea 8: syntax error
  -> INVALIDA (error sintactico en la linea 8)
[9] 10 / ()
  [Error] linea 9: syntax error
  -> INVALIDA (error sintactico en la linea 9)
[10] x + (y * ) - 2
  [Error] linea 10: syntax error
  -> INVALIDA (error sintactico en la linea 10)

--- Resumen ---
Expresiones validas  : 0
Expresiones invalidas: 10
```

Figure 3: Salida del analizador sobre incorrectas.txt.

Table 2: Errores detectados.

Línea	Expresión	Tipo	Causa
1	`2 + * 3`	Sintáctico	Dos operadores seguidos
2	`(2 + 3 * 4`	Sintáctico	Falta cerrar el paréntesis
3	`2 + 3) * 4`	Sintáctico	Paréntesis de cierre sin abrir
4	`5 5`	Sintáctico	Dos operandos sin operador
5	`* 7 + 1`	Sintáctico	Empieza con operador binario
6	`3 +`	Sintáctico	Falta el segundo operando
7	`4 $ 2`	Léxico y sintáctico	Carácter `$` no reconocido
8	`2valor + 1`	Léxico y sintáctico	Identificador mal formado
9	`10 / ()`	Sintáctico	Paréntesis vacíos
10	`x + (y * ) - 2`	Sintáctico	Falta un operando dentro del paréntesis

Análisis de los resultados

El analizador aceptó todas las expresiones correctas y rechazó todas las incorrectas. Los errores léxicos (líneas 7 y 8) los detectó Lex antes de que Yacc recibiera la cadena, por lo que en esos casos se imprimen dos mensajes: el léxico, que dice cuál es el carácter o lexema problemático, y el sintáctico, porque el token

de error que Lex envía no encaja en la gramática. Los errores sintácticos los detectó Yacc en el momento en que llegó un token que ninguna producción permitía en ese punto (por ejemplo, el `*` de la línea 1 justo después de un `+`). Gracias a la regla `error` el análisis se recuperó en cada salto de línea y se reportaron las 10 líneas en una sola pasada.

Una limitación es que el mensaje de Bison es genérico (`syntax error`) y no dice qué token se esperaba; se podría mejorar con `%define parse.error verbose`. Otra es que las expresiones con variables solo se validan, no se evalúan, porque el programa no tiene tabla de símbolos.

7. Cuestionario

1. **¿Qué tipo de errores fueron detectados en los ejemplos incorrectos y cómo influyeron las gramáticas definidas para lograr su identificación?**

   Se detectaron errores léxicos (el carácter `$` y el identificador `2valor`), que no pertenecen a ningún token, y errores sintácticos: operadores seguidos, operandos sin operador, expresiones incompletas y paréntesis desbalanceados o vacíos. Estos últimos se detectan porque la gramática define exactamente en qué orden pueden aparecer los tokens: una `expresion` solo puede ser un número, un identificador, algo entre paréntesis o dos expresiones unidas por un operador, así que cuando llega un token que no encaja en ninguna producción, Yacc sabe que la cadena no pertenece al lenguaje. En particular, el balance de los paréntesis no se puede comprobar con una expresión regular y se logra gracias a la recursión de la gramática.

2. **¿Cómo contribuye el uso de derivaciones por la izquierda o por la derecha en la construcción de un árbol sintáctico para validar las operaciones y las prioridades de las operaciones?**

   La derivación indica en qué orden se expanden los no terminales hasta llegar a la cadena, y cada forma de derivar produce un árbol. Para `2 * 3 + 4`, si la gramática obliga a que el `+` se expanda primero (queda en la raíz) y el `*` después (queda más abajo), el producto se calcula antes que la suma. Por eso la prioridad se logra escribiendo la gramática por niveles, o con `%left` y `%right` en Yacc. Yacc hace un análisis ascendente que reconstruye una derivación por la derecha, y la asociatividad se refleja en cómo se reduce: con una regla recursiva por la izquierda (`E -> E - T`) se obtiene `(8 - 3) - 2`, y con una recursiva por la derecha (`F -> B ** F`) se obtiene `2 ** (3 ** 2)`.

3. **¿Qué ajustes serían necesarios en la gramática para permitir el reconocimiento de estructuras más complejas, como (a+b)^2, raíz cuadrada, así como operaciones trigonométricas?**

   Para `(a+b)^2` habría que agregar `^` como token de potencia (el `**` ya está) con una regla `expresion '^' expresion` y mayor prioridad que la multiplicación, asociando por la derecha. Para la raíz cuadrada y las funciones trigonométricas habría que agregar tokens para los nombres (`sqrt`, `sin`, `cos`, `tan`) en Lex, antes de la regla de identificadores para que no se tomen como variables, y una regla en Yacc del tipo `expresion: SQRT '(' expresion ')' | SIN '(' expresion ')' | ...`. En una gramática más completa también se podrían agregar funciones con varios argumentos separados por coma, y una tabla de símbolos para guardar las variables.

8. Conclusiones

Con esta práctica entendí cómo se complementan el analizador léxico y el sintáctico: Lex se encarga de los lexemas y Yacc de su orden, y entre los dos se puede validar una expresión completa con muy poco código. Lo que más me costó fue la precedencia y la asociatividad, porque si no se declaran bien los operadores Bison avisa de conflictos y `2 + 3 * 4` puede dar 20 en lugar de 14; con `%left`, `%right` y `%prec` se resolvió y fue más sencillo que reescribir la gramática por niveles. El WOW fue ver que con la regla `error '\n'` el analizador no se rinde en el primer error y reporta las 10 líneas incorrectas en una sola corrida. El WTF fue un fallo mío: al principio todos los resultados salían como números absurdos (como `9.88e-324`) porque `yylval` era un `int` en el archivo de Lex y un `double` en el de Yacc; había que definir el mismo `YYSTYPE` en los dos. También noté que el mensaje por defecto de Bison, `syntax error`, es poco útil, y que mejoraría con `parse.error verbose`. En general, la práctica me sirvió para ver cómo un analizador sintáctico de un compilador valida la estructura de lo que escribe el programador.

Referencias Bibliográficas

References

[1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (**2007**). *Compilers: Principles, Techniques, and Tools*. Segunda edición. Pearson Addison-Wesley. U.S.A.

[2] Giró, J., Vázquez, J., Meloni, B., Constable, L. (**2015**). *Lenguajes formales y teoría de autómatas*. Alfaomega. Argentina.

[3] Hopcroft, J. E., Motwani, R., Ullman, J. D. (**2007**). *Introducción a la teoría de autómatas, lenguajes y computación*. Pearson Educación. México.

[4] Levine, J. (**2009**). *Flex & Bison*. O'Reilly Media. U.S.A.